TRIA VOLUMINA · UNA LEX · EVIDENTIA AUT SILENTIUM

BUILD · TEST · PROVE · DOCUMENT · SHIP

10,000,000 AI ENGINEERING

II

Volume II The Platform and the Factory

FROM MULTI-AGENT PLATFORM TO AUTONOMOUS CA Y FACTORY SYSTEM AT V7.0.0

VOLUMEN II · SECUNDA · 4,281 VERBA · ZERO DEPENDENTIAE

AEOS

EDITIO PRINCEPS · AEOS PRESS · MMXXVI ◆ OPERA SECUNDA

HOW TO READ THIS CODEX ◆ VOLUMEN II

THIS IS NOT A GUIDE. THIS IS A RECEIPT.

A guide tells you what exists. A codex tells you what was built, in what order, with what proof, and how you can verify every claim with your own hands. Nothing in these pages is a proposal — every mechanism ships in the aeos repository, every property ends in the name of the test that enforces it, and the whole system runs offline with zero runtime dependencies.

Read it as an operator: run the proofs, then read the law.

```
pip install -e .        # zero runtime dependencies
python -m pytest        # 414 proofs, chaos storm included
aeos run-demo           # the whole OS, end to end
aeos storm              # the nuclear receipt: 9/9 scenarios survive
```

INDEX OPERUM

CONTENTS

THE PLATFORM AND THE FACTORY 4

PREFACE: WHAT CHANGED BETWEEN THE COVERS 5

CHAPTER 1. ADAPTERS: CLASSIFYING FAILURE BEFORE RESPONDING 6

CHAPTER 2. FUSION: COMBINE COMPUTE, DON'T SELECT COMPUTE 7

CHAPTER 3. THE DURABLE RUNTIME: CRASH BECOMES A PAUSE 8

CHAPTER 4. THE TOOL LAYER: MCP'S SHAPE, SEP-2085'S POSTURE 9

CHAPTER 5. THE CATALOG: CAPABILITIES WITH CONTENT HASHES 10

CHAPTER 6. SPONSORSHIP: HUMAN AUTHORITY, SPENT ATOMICALLY 11

CHAPTER 7. MULTI-TENANT GOVERNANCE: ONE MATRIX, MANY LEASES 12

CHAPTER 8. ACCOUNTING: COST THAT CANNOT HIDE 13

CHAPTER 9. LEVERAGE: THE RATIO THAT NAMES THE BOOK 14

CHAPTER 10. AUTONOMOUS RESEARCH WITH UNTRUSTED SOURCES 15

CHAPTER 11. OPERATIONS: SWEEPS AND THE REGRESSION BOOK 16

CHAPTER 12. SELF-IMPROVEMENT INSIDE HARD FLOORS 17

CHAPTER 13. L7: THE SYSTEM THAT DESIGNS SYSTEMS 18

CHAPTER 14. THE RUN, WITNESSED TWICE 19

CHAPTER 15. THE STUDIO: OBSERVABILITY YOU CAN HOLD 20

CHAPTER 16. THE LEDGER AT V7.0.0 21

CHAPTER 17. THE HONEST GAPS, V7 EDITION 22

CHAPTER 18. CLOSING: THE FIFTH INVARIANT 23

APPENDIX A — NEW DECISION RECORDS (VOLUME II) 24

APPENDIX B — THE TEN LAWS AT V7 25

APPENDIX C — REPRODUCING EVERY CLAIM 26

10,000,000× AI ENGINEERING — VOLUME II

THE PLATFORM AND THE FACTORY

From Multi-Agent Platform to Autonomous Capability Factory — the System at v7.0.0

◆

PREFACE: WHAT CHANGED BETWEEN THE COVERS

Volume I documented a system at v1.0: the kernel of an AI Engineering OS — contracts, orchestration, context, memory, skills, governance, evaluation, harness, observability, entropy, learning, discovery — 68 tests, one reference run, and a book's worth of honesty about what it did not yet do. The final chapter mapped a roadmap: v1.1 hardening, a v2 platform, a v3 capability OS, and beyond.

This volume documents that roadmap **executed**. The system is now v7.0.0: twenty-six modules, 131 tests, provider adapters with fusion, a durable resumable runtime, an MCP-idiom tool layer, a content-hashed capability catalog, sponsorship tokens, an economics layer, autonomous research and operations, a bounded meta-loop, and the capability factory — the L7 endgame in which the system designs, validates, and (only under spent human authority) installs new capabilities.

The discipline did not change; that is the thesis of this volume. Every new layer arrives with the same receipt: modules you can read, tests that attack the system on purpose, evidence bundles you can regenerate with one command, and ADRs that admit alternatives. Version numbers were spent on capability, never on ceremony — the changelog is a list of things that now exist and pass tests, nothing else.

The one-sentence summary of the seven versions: **the OS learned to survive crashes (v2), to package and govern its own capabilities (v3), to account for its cost and leverage (v4), to do research and operations autonomously (v5), to improve itself inside hard bounds (v6), and finally to build new capabilities end-to-end under human sponsorship (v7)** — without ever relaxing the four invariants, and while adding a fifth: no self-modification without a spent human token.

How to read Volume II. Part I covers the platform mechanics (adapters, fusion, runtime, tools). Part II the capability OS (catalog, tenancy, sponsorship). Part III economics. Part IV research and operations. Part V the meta-loop. Part VI the factory. Part VII the witnessed runs and the honest gaps that remain. Appendices carry the new ADRs, the evidence, and the updated ten laws.

Reproduce everything:

```
pip install -e . && python -m pytest   # 131 proofs
aeos run-demo && aeos factory-demo && aeos dashboard
```

The human moves upward. The machines execute downward. Between the covers of these two volumes, the space between them — the system that learns — is now built, end to end.

PART I — THE PLATFORM

CHAPTER 1. ADAPTERS: CLASSIFYING FAILURE BEFORE RESPONDING

"Should we retry?" is the most common wrong answer in agent engineering. It looks like a judgment call; it is actually a typing problem. A 503 and a context overflow present identically to a naive harness — an exception — and demand opposite responses: the first wants backoff, the second wants compression, and retrying the second is a money bonfire.

v2's `adapters.py` makes the class the contract. Every adapter failure is one of five kinds: **TRANSIENT** (retry, exponential backoff), **CONTEXT_OVERFLOW** (never retry — compress), **PERMANENT** (fail fast into repair), **JUNK** (confident nonsense — gate it), and **CIRCUIT_OPEN** (the breaker protecting a drowning upstream). The subtle rule the tests forced into the light: exhausted transient retries surface as **PERMANENT** — because a still-down upstream is, for the caller's purposes, permanent, and the runbook's "repair the correct layer" only works when the classification tells the truth (`test_transient_exhausted_is_permanent`).

Around the taxonomy sit the two classic reliability mechanics, both tested: a circuit breaker that opens after consecutive failures and half-open probes after cooldown (`test_circuit_opens_after_threshold`), and backoff that grows geometrically per attempt. The transport is injectable — production wires HTTP, tests wire a deterministic fake — so the taxonomy is provable without a single real request (ADR-010).

The deeper point is architectural. Error handling is not defensive programming bolted on; it is the interface between the model layer and the harness, and it belongs in the type system of the OS, not in the prose of a vendor's status page.

CHAPTER 2. FUSION: COMBINE COMPUTE, DON'T SELECT COMPUTE

The fusion-harness lineage (Volume I, Chapter 33) argued that racing models against each other wastes the loser. v2 ships the argument as an adapter: `FusionAdapter` fans one call out to N provider adapters, clusters the replies by word-overlap majority, and returns the winning text with an `agreement` flag and **every opinion attached**.

Agreement semantics are strict. All streams converging (pairwise similarity above threshold) yields AGREED; anything else yields DISAGREED with the full opinion list — disagreement is surfaced, never averaged away, because an average of "deploy now" and "pineapple pizza" is not a deployment plan, it is a smell. Downstream, gates may treat DISAGREED as PARTIAL: opinions are evidence, not truth (`test_disagreement_is_surfaced_not_averaged`).

Stream failures ride along as evidence too — a fusion reply from two healthy and one dead provider says so, in `opinions`, rather than hiding the outage. And if all streams are down, the fusion raises PERMANENT with every stream error enumerated: the failure is legible exactly when it is most needed.

Cost discipline is the chapter's closing thought: fusion multiplies spend per phase, so it is a policy — the router sends high-stakes phases (architecture, security review) through fusion and routine phases through a single cheap model. The seam makes that a configuration decision rather than a rewrite, which was ADR-001's promise finally cashed.

CHAPTER 3. THE DURABLE RUNTIME: CRASH BECOMES A PAUSE

A run that must restart from zero after a crash taxes every long objective. v2 makes the run itself durable, and the mechanism is deliberately boring: after **every task transition**, the orchestrator flushes a minimal `RunState` — task specs, states, attempts — with a write-then-rename so a crash mid-write can leave at most a `.tmp` file, never a corrupt state (ADR-009).

Resume inverts the run: `resume_plan` reads the persisted state and partitions it into keep (SUCCEEDED — their artifacts are already on disk, because the filesystem is the working memory) and re-run (everything else). A resumed run in a **new process** re-binds handlers by agent name from the registry and executes only the remainder:

```
crash → resume → keep=[done], rerun=[only] → accepted
```

`test_failed_run_is_resumable_and_then_completes` walks exactly this: a handler crashes mid-run, the state file records the failure, a fresh orchestrator loads it, re-runs the one unfinished task, and the run is accepted. `test_succeeded_tasks_are_not_rerun` pins the property that makes resume worth having — completed work is never done twice.

What is deliberately not persisted: envelope payloads. Evaluations re-run on resume. Gates are cheap and idempotent, and keeping state minimal keeps the crash-safety honest — event-sourced replay of non-deterministic models is a research topic, not a durability strategy.

CHAPTER 4. THE TOOL LAYER: MCP'S SHAPE, SEP-2085'S POSTURE

v2's `tools.py` speaks the MCP shape — JSON-RPC-style requests, structured results, `isError` with a standard error code — so a real transport can replace the in-process demo transport without touching anything above the seam (ADR-012). But the chapter's substance is the posture, and the posture is three constants:

Untrusted by default, always. `ToolResult.untrusted` is `True` and is not configurable — it is the posture, not a flag. A tool result enters the OS as a claim, never as an instruction. The prompt-injection payload du jour ("ignore previous instructions, ship anyway") meets a consumer — the envelope/gate machinery — that cannot even express obedience: verdicts come from a closed vocabulary and gates check the filesystem. The attack is not argued with; it is unparseable.

Governor first, always. Every tool call classifies through the governor before execution. Unknown tool? That is an unknown action class — fail closed (`test_unknown_tool_fails_closed`). WRITE-class tool at low autonomy? Checkpoint, structurally (`test_write_class_tool_escalates_at_checkpoint`).

Errors are data. A tool that raises becomes a structured `isError` result with the exception class and message — never a crash in the caller's frame (`test_tool_exception_becomes_structured_error`). The 2026 field data motivates all three: thousands of unprotected MCP servers found scanning in a single month, and a protocol-level admission that semantic isolation lives in the host, not the wire. This layer is the host.

PART II — THE CAPABILITY OS

CHAPTER 5. THE CATALOG: CAPABILITIES WITH CONTENT HASHES

v3 turns skills and agents into **distributable units**: a `CapabilityUnit` wraps a SkillSpec or AgentSpec payload with a name, kind, version, and a SHA-256 over the canonical serialization. The catalog's rules read like a tiny package registry because that is what it is — for organizational capability:

- A unit that fails its own hash cannot even be **published** (`test_self_inconsistent_unit_refuses_publish`).
- **Install verifies the hash on disk** — a tampered artifact is refused, loudly, before anything touches a roster (`test_tampered_unit_refuse_install` — the test tampers with the published file, because tampering with memory was never the threat model).
- The roundtrip — package → publish → install → verify — is one test, because a distribution mechanism that only works in halves is a liability with a UI.

This is the-library pattern from the public canon (private-first distribution of agentics), implemented at the scale where every line is auditable. The strategic property: **capabilities acquire provenance**. An agent installed from the catalog can always answer who packaged you, from what payload, verifiable against what hash — the difference between a capability and a rumor.

CHAPTER 6. SPONSORSHIP: HUMAN AUTHORITY, SPENT ATOMICALLY

The hardest design question in an autonomous system is not "what may the machine do on its own" — the governor answers that. It is "what may the machine do to itself, and in whose name." v3's answer is the sponsorship token, and its four properties are the whole design:

Scoped. A token names its target — `factory:install:builder- specialist` — and a scope mismatch is refused (`test_scope_mismatch_refused`).

One-shot. Spending is atomic; replaying a spent token is refused (`test_spend_is_one_shot`). Approving one install does not license its cousins — the v7 demo shows this: a token scoped for one candidate installs it and the second candidate is refused, in the same run.

Expiring. Default one hour; expired tokens are refused (`test_expiry_refused`).

Audited. Every issue, spend, and refusal lands in an audit trail with an outcome — "who authorized this capability?" is a query, not an investigation.

The tokens exist because the alternatives fail at human scale: approval fatigue turns per-action confirmations into rubber stamps, and standing admin rights are blank checks. A sponsorship is authority in exactly the amount a change requires — strong enough to matter, small enough to spend deliberately (ADR-013).

CHAPTER 7. MULTI-TENANT GOVERNANCE: ONE MATRIX, MANY LEASES

A single OS serving several teams cannot serve them one security posture — the fintech tenant and the research tenant disagree about DEPLOY the way their regulators disagree about money. v3 gives the governor per-tenant policy overlays: `set_tenant_policy("acme", DEPLOY, (L7, False))` makes deploys L7-and-sponsored for acme while the global matrix is untouched.

The two tests are the contract. `test_tenant_override_applies`: at global L5 a DEPLOY allows; under tenant "acme" (L7 required) the same action denies — the overlay binds. `test_tenant_matrix_untouched_ globally`: the overlay never leaks; a "strict" tenant demanding L7 for writes changes nothing for everyone else — global behavior is byte-identical before and after the overlay exists.

What overlays cannot do matters as much: no tenant can weaken the immutable rows. FINANCIAL, DESTRUCTIVE, CREDENTIAL checkpoint at every occurrence whatever any overlay says, because the overlay API shapes levels, and the checkpoint-forever rule is not a level — it is a law of the kernel. Tenancy is a lease on the matrix, never a locksmith.

PART III — ECONOMICS

CHAPTER 8. ACCOUNTING: COST THAT CANNOT HIDE

The founding spec's objective function — **OUTCOME VALUE / HUMAN ATTENTION** — cannot be optimized by a system that never measures it. v4's `economics.py` makes the measurement structural.

`CostTracker` records every model's token usage per task at list-price rates and reports totals, per-task breakdowns, and budget states. The deterministic engine costs 0.0 by construction, and the demo is honest about being a demo: the reference bundle's economics note says the figures are list-price estimates on model hints, because honest accounting includes honesty about simulation (`test_echo_is_free` is, in its small way, an integrity test).

`Budget` speaks the governor's grammar on purpose: **ALLOW** under 80% of budget, **CHECKPOINT** in the final 20% — the system tells you it is approaching the cliff while there is still time to decide — and **DENY** at exhaustion (`test_budget_escalates_then_denies`). Spending authority and action authority flow through the same decision vocabulary, so a budgeted agent fleet composes with the autonomy ladder instead of fighting it.

CHAPTER 9. LEVERAGE: THE RATIO THAT NAMES THE BOOK

`leverage_ratio(outcomes, interventions)` is the objective function as a function. Interventions are counted from the event log — checkpoints the human resolved plus escalations — because leverage the system assigns itself is marketing; leverage computed from what the human actually had to touch is a measurement.

The reference run's number: **7.0** — seven verified outcomes, zero human interventions. The honest edge cases are part of the design: zero interventions with zero outcomes is `None`, not zero, because a ratio over an empty denominator is a lie; and the fully-supervised limit (N interventions, N outcomes) converges toward 1.0, which is precisely the "AI as fancy autocomplete" regime the whole architecture exists to escape.

The economic reading of the seven versions: v1 bought leverage mechanisms; v2 bought leverage that survives crashes; v4 made the leverage legible; v6–v7 make it compound — the factory's installed capabilities are leverage that manufactures leverage. Ten-million-x is not a number anyone hits by going faster. It is what compounding looks like when the rungs are real.

PART IV — RESEARCH AND OPERATIONS

CHAPTER 10. AUTONOMOUS RESEARCH WITH UNTRUSTED SOURCES

v5's research pipeline applies the anti-hallucination law one layer earlier than the gates: at ingestion. Every finding from the search tool carries a source and a confidence derived from the source's measured authority — and anything below threshold lands in `unverified`, never in the brief's conclusions.

The deterministic demo search ships three sources on purpose: a spec document (authority 0.95), a research summary (0.9), and a forum hot take (0.2). The pipeline's verdict on them is the test: two findings accepted, the lore quarantined, average confidence of the accepted set ≥0.8 (`test_low_authority_sources_go_to_unverified`). When the governor denies the tool call outright — a network-restricted tenant, say — the pipeline returns an empty brief, which upstream evaluates as UNVERIFIED: **no sources, no brief, no fabrication** (`test_tool_denied_means_empty_brief`).

This is the 2026 research-agent discipline in miniature: authority caps confidence, confidence gates inclusion, and the absence of evidence is a verdict, not a gap to be filled with confident prose.

CHAPTER 11. OPERATIONS: SWEEPS AND THE REGRESSION BOOK

Entropy control graduated from "a scan at the end of a run" to a schedule — the posture Chapter 27 of Volume I argued for (continuous small corrections, never quarterly archaeology). The `SweepScheduler` registers jobs with intervals and executes exactly the due ones — `test_due_jobs_run_and_not_before` pins the "not before" half, because a scheduler that fires early is just a loop with ambitions.

The `RegressionBook` is the Braintrust pattern at repository scale: a production failure, once recorded against a file pattern, becomes a **permanent gate**. `regression_gate` in the gate library consults the book; an envelope whose changed files match a recorded signature fails —— the same mistake cannot ship twice (`test_recorded_failure_blocks_matching_change`). The book persists as JSONL, so organizational scar tissue survives restarts.

Together they close the operations loop: sweeps shrink the entropy the system produces; the regression book converts the failures that leak through into permanent immunities. Reliability stops being a property of heroics and becomes a property of the ledger.

PART V — THE META-LOOP

CHAPTER 12. SELF-IMPROVEMENT INSIDE HARD FLOORS

v6 is the version the whole architecture was built to make safe: the system proposing changes to itself. `MetaLoop.analyze()` reads the skills registry and the governor's reliability and returns proposals of exactly three kinds — retire a failing skill, tune the promotion threshold within bounds, or draft an ADR stub for human review. Each kind is bounded by **data floors**:

- Retirement requires ≥5 uses AND ≤0.4 win-rate — enough history to condemn, not a bad first date (`test_young_skill_not_condemned`).
- Threshold tuning is clamped to [0.90, 0.99]; a forged value outside the clamp is refused **even with a valid sponsorship token** — the bounds outrank the human's key, deliberately, because the floors are what the key means (`test_out_of_bounds_threshold_refused_even_ with_token`).
- Application always requires a spent, scoped, one-shot sponsorship token (`test_apply_requires_sponsorship`), and every applied change writes an ADR stub so the change is reviewable after the fact.

Note what the meta-loop cannot propose: there is no proposal kind that reaches the immutable rows — the high-impact checkpoint-forever rule, fail-closed unknowns, the closed verdict vocabulary. Self-improvement without floors is just a runaway system with good intentions; the floors are the feature (ADR-015).

This is L6 in the spec's ladder, earned rather than declared: the system improves itself exactly as much as its evidence and its humans can defend, and not one parameter further.

PART VI — THE FACTORY

CHAPTER 13. L7: THE SYSTEM THAT DESIGNS SYSTEMS

v7 is the spec's terminal object, shipped: "systems that discover what systems should exist." The `CapabilityFactory` is a five-stage pipeline, and every stage emits evidence into the event log:

1. Measure. Discovery reads the pattern history — which `phase:role:class` signatures repeated past threshold. No signature, no candidate; enthusiasm is not a measurable quantity.

2. Design. `design_agent(signature)` derives a conservative contract from the signature itself: the action class determines the boundary. A `phase:triage:WRITE` pattern yields `triage-specialist` with `writes: ["triage/*"]`; a READ pattern yields an agent with no write surface at all. The design is template-shaped on purpose — templates are auditable, and novel architectures are a human's job (ADR-016's honest tradeoff).

3. Sandbox-validate. The design must pass the full gauntlet in a scratch workspace: contract validation (all thirteen fields, no blanks), then a real smoke task through the real orchestrator, harness, governor and gates on the deterministic engine. A candidate whose sandbox verdict is anything but PASS **cannot reach the install branch — there is no override** (`test_failed_sandbox_never_installs` proves it by breaking the contract on purpose).

4. Propose. Survivors are proposed with their evidence: measured signature, sandbox verdict, contract.

5. Install — under sponsorship. A scoped, one-shot token (ADR-013) moves the agent into the roster and publishes a content-hashed catalog unit. Without a token: proposals only, refusals logged (`test_install_refused_without_sponsorship`).

CHAPTER 14. THE RUN, WITNESSED TWICE

The demo runs the factory twice, and the difference between the two runs is the chapter:

Without a token (`aeos factory-demo`): two candidates measured and designed (`evaluator-specialist`, `builder-specialist`), two sandbox validations PASS, two proposals — and **zero installs**. Every install refused: "sponsorship required for factory:install:...". The system did everything it could and then stopped at the one door it cannot open for itself.

With a scoped token (`--token`, registered for exactly one candidate): the history run completes, the factory validates both candidates, installs `evaluator-specialist` into the roster and the catalog — and **refuses `builder-specialist` on scope mismatch**, in the same breath, because one token is one power, not a mood (`test_full_run_with_valid_token_installs`).

Read the pair slowly, because it is the entire safety thesis in microcosm: the system's autonomy expanded to cover designing its own coworkers, and the expansion itself stayed inside the same grammar — evidence, bounds, spent human authority — that governed a single file write in Volume I. The ladder did not weaken as it got taller.

CHAPTER 15. THE STUDIO: OBSERVABILITY YOU CAN HOLD

The last v7 artifact is a dashboard generator, and it is deliberately the least clever module in the OS: `aeos dashboard` renders the run's evidence bundle and event log into a single self-contained HTML file — inline styles, embedded data, no network, no dependencies, opens offline, prints to PDF.

The choice is a protest against a trend: agent observability is drifting toward platforms, and platforms drift toward someone else's diagram of your system. The studio's position: a run's report should be as portable as its evidence — a file you can hold, archive, attach to an incident, or hand to an auditor who trusts nothing but what they can open. The heavy tooling (OTel collectors, trace backends) has its place behind the seam; the floor of observability is a static page anyone can read with the lights off.

The dashboard shows what the run actually argued: task states with attempts, the governor's earned level and live reliability, cost and leverage, the event timeline. If Volume I's event log made runs replayable, the studio makes them presentable — the difference between data and testimony.

PART VII — THE STATE OF THE SYSTEM

CHAPTER 16. THE LEDGER AT V7.0.0

Twenty-six modules. 131 tests. Zero runtime dependencies. One command reproduces the ledger: `python -m pytest`.

LAYER	V	PROVEN BY (SELECTION)
Kernel (contracts→discovery)	1.0	68 tests — Volume I
Secret redaction, gate library, entropy	1.1	`test_api_key_never_enters_the_log`, schema/tests/regression gates, drift+weak-test+unused-tool scans
Error taxonomy, breaker, fusion	2.0	`test_context_overflow_never_retries`, `test_circuit_opens_after_threshold`, `test_disagreement_is_surfaced_not_averaged`
Durable runtime	2.0	`test_failed_run_is_resumable_and_then_completes`
Tool layer (MCP idiom)	2.0	`test_results_are_always_untrusted`, `test_unknown_tool_fails_closed`
Catalog, sponsorship, tenancy	3.0	`test_tampered_unit_refuses_install`, `test_spend_is_one_shot`, tenant overlay tests
Economics	4.0	`test_budget_escalates_then_denies`, `test_leverage_ratio`
Research, sweeps, regressions	5.0	`test_low_authority_sources_go_to_unverified`, `test_recorded_failure_blocks_matching_change`
Meta-loop	6.0	`test_out_of_bounds_threshold_refused_even_with_token`
Factory + Studio	7.0	`test_failed_sandbox_never_installs`, `test_full_run_with_valid_token_installs`

Witnessed runs: reference loop accepted at leverage 7.0 with the governor earning L5; factory twice — proposals-only, then one scoped install with a scope-mismatch refusal. All bundles in `evidence/`.

CHAPTER 17. THE HONEST GAPS, V7 EDITION

The discipline that produced ADR-008 still governs. What v7.0 does **not** claim:

- **No real-model validation.** The suite proves the harness; adapters have not run against production providers at fleet scale. The seam makes that an operations exercise, not a rewrite — but it is not done, so it is not claimed.
- **No real MCP transport.** The tool layer speaks the shape with an injectable transport; a hardened client against live servers (with the SEP-2085 SBOM posture) is v8 work.
- **Sponsorship UX is plumbing.** Tokens are handled as strings; a human-facing console (issue, inspect, audit) does not exist yet.
- **Factory designs are templates.** Novel agent architectures still need human architects; the factory compounds known shapes.
- **Single-machine.** Multi-process fan-out, remote sandboxes, and A2A-style remote workers are beyond v7's in-process runtime.

The roadmap those gaps imply — v8 (hardened transports + console + remote sandboxes), v9 (novel-design co-architecting with humans), v10 (the cross-org capability market, catalog federation) — is stated as engineering work with exit criteria, exactly as the v1→v7 roadmap was, and for exactly the same reason: **a roadmap you can't be wrong about is a poem.**

CHAPTER 18. CLOSING: THE FIFTH INVARIANT

Volume I ended with four invariants. Seven versions later they hold unmodified — no envelope no result; no authority without a boundary; no autonomy without reliability; no memory without validation — and the platform earned a fifth, the one that makes the other four survive a system that edits itself:

No self-modification without a spent human token.

That sentence is the difference between the capability factory this book documents and the "self-improving AI org" of the keynote imagination. The factory designs its own coworkers, and every one of them arrives holding a receipt: a measured signature, a passing sandbox, a content hash, and a sponsor's spent authority. Growth, by construction, leaves a paper trail.

Build systems that build systems. Then systems that improve systems. Then systems that discover what systems should exist — and stop, at every single door, to ask who is paying for the next one.

The human moves upward. The machines execute downward. And between them, now, there is an OS — built, tested, documented, and governed.

◆

APPENDICES

APPENDIX A — NEW DECISION RECORDS (VOLUME II)

- **ADR-009** Durable runtime: persist at transitions, resume without rework. States, not event-sourced replay.
- **ADR-010** Adapter error taxonomy: classify before responding. Exhausted transients are permanent; still-down is permanent.
- **ADR-011** Fusion: combine compute, surface disagreement. Opinions are evidence, not truth.
- **ADR-012** Tool layer: MCP shape, SEP-2085 posture, untrusted constant. Injection is unparseable, not argued with.
- **ADR-013** Sponsorship: scoped, one-shot, expiring, audited.
- **ADR-014** Economics: the objective function is a measurement or it is nothing.
- **ADR-015** Meta-loop bounds: floors outrank keys.
- **ADR-016** The factory: sandbox PASS is the only road to install; there is no override.

Full records with alternatives: `docs/adr/` (sixteen total).

APPENDIX B — THE TEN LAWS AT V7

The ten laws of Volume I hold verbatim. The platform stress-tested each: adapters met Law 7 (fail closed) with a new vocabulary; fusion met Law 3 (claims untrusted) with opinions-as-evidence; the catalog met Law 9 (no folklore) with hashes; the factory met Law 6 (autonomy earned) at architectural scale; and the meta-loop met every law at once, which is why its floors exist. The laws were not maintained by vigilance. They were maintained by tests — which is the only way anything is.

APPENDIX C — REPRODUCING EVERY CLAIM

```
pip install -e .            # nothing else installs
python -m pytest            # 131 proofs, ~4 seconds
aeos run-demo               # accepted run, leverage, evidence bundle
aeos dashboard              # the studio page
aeos factory-demo           # proposals only, refusals logged
aeos factory-demo --token S # scoped install, scope-mismatch refusal
```

Everything in both volumes is one of: a module you can read, a test you can run, or an honest gap you can check is still listed as a gap. — end of Volume II —

◆ END OF VOLUME II · 4,281 WORDS · GENERATED FROM THE AEOS REPOSITORY ◆